



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

FPGA-based heterogeneous systems & AMD Vitis AI

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

FPGA-based heterogeneous systems

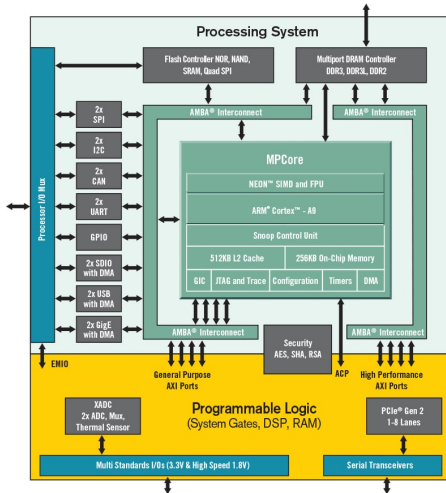
Un sistema heterogéneo está compuesto por dos o más unidades de procesamiento de distinto tipo. En este trabajo se considera la combinación de:

- ▶ CPU: procesamiento general.
- ▶ FPGA: lógica programable para aceleración.

- ▶ Liberar al CPU de carga de cómputo intensiva.
- ▶ Acelerar operaciones específicas mediante paralelización en FPGA.

Los SoC modernos (ej. familia Zynq de AMD) integran CPU y FPGA en un mismo chip, facilitando la construcción de sistemas heterogéneos compactos y eficientes.

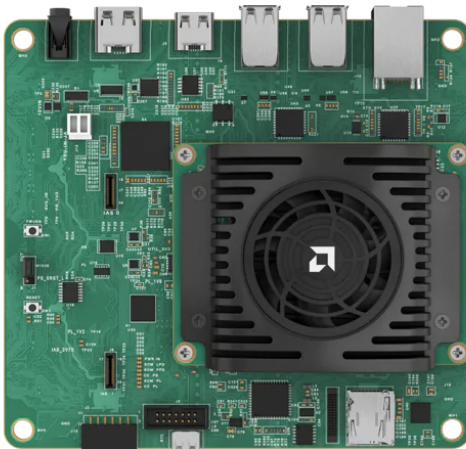
AMD Xilinx Zynq System-on-chip



Zynq, es un SoC de Xilinx que integra un procesador ARM Cortex-A (Processing System, PS) junto con textblógica programable (Programmable Logic, PL).

AMD / Xilinx Kria™ KV260 Vision AI Starter Kit

Out-of-the-box platform for advanced vision application development without requiring complex hardware design knowledge. The KV260 starter kit is equipped with a non-production version of the production AMD Kria K26 SOM. This SOM and fansink are mounted to an evaluation carrier card optimized for vision applications, featuring multi-camera support via ON Semiconductor Imager Access System (IAS) and Raspberry Pi connectors.



AXI: Advanced eXtensible Interface

AXI (Advanced eXtensible Interface) es un protocolo de interconexión de alto rendimiento, parte de la especificación **AMBA** (Advanced Microcontroller Bus Architecture) desarrollada por ARM.

- ▶ Diseñado para ofrecer alta eficiencia en comunicación entre **procesadores**, **memorias** y **periféricos** en sistemas SoC y FPGA.
- ▶ Soporta múltiples canales independientes: *Read Address*, *Write Address*, *Write Data*, *Read Data*, y *Write Response*.
- ▶ Permite transferencias de datos **out-of-order**¹, acceso burst (rafagas de datos) y canales pipelined (e.g. flip flops, throughput).
- ▶ Facilita la integración modular de IP cores, especialmente en plataformas como Xilinx Zynq o Intel SoC-FPGA.

Aplicaciones

Interfaces entre CPU y controladores DMA², conexiones a memoria externa, buses de interconexión interna, diseño de periféricos personalizados.

¹respuestas pueden llegar en un orden diferente al que fueron solicitadas, lo que permite optimizar la latencia del sistema

²Direct Memory Access

Host Compilation vs. Cross-Compilation

Host Compilation



Compiling application for execution on same platform

Example: Visual Studio or NetBeans

- Desarrollo y ejecución ocurren en la misma plataforma.

Host Compilation vs. Cross-Compilation

Cross-Compilation



Target

- Development and target environment differ
- Example: Development for Arm® processor using Windows
- Cross-compilation frequently used for embedded systems

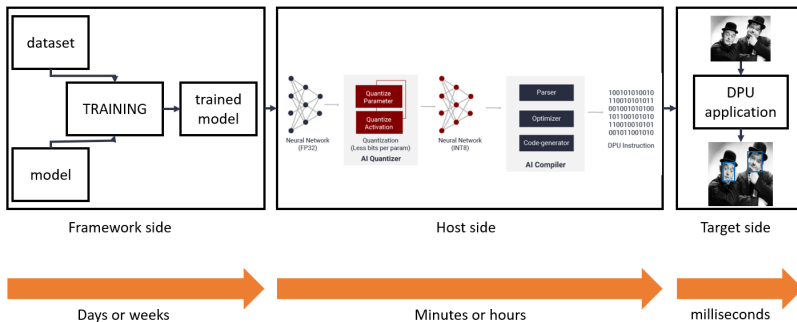
Vitis™ Unified IDE:

- Runs on Windows/Linux® OS
 - Example: AMD x86 processor
- Targets:
 - MicroBlaze™ and Arm architectures within an SoC device

- **Compilación en una plataforma (host) para ejecutarse en otra diferente (target).**
- Permite desarrollar apps para arquitecturas distintas (ej: x86 para ARM).
- No requiere que el sistema operativo y el procesador del host coincidan con el del target.

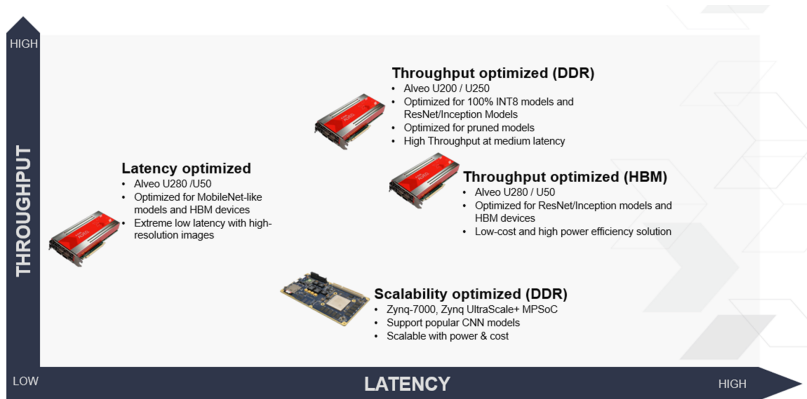
AMD Vitis AI

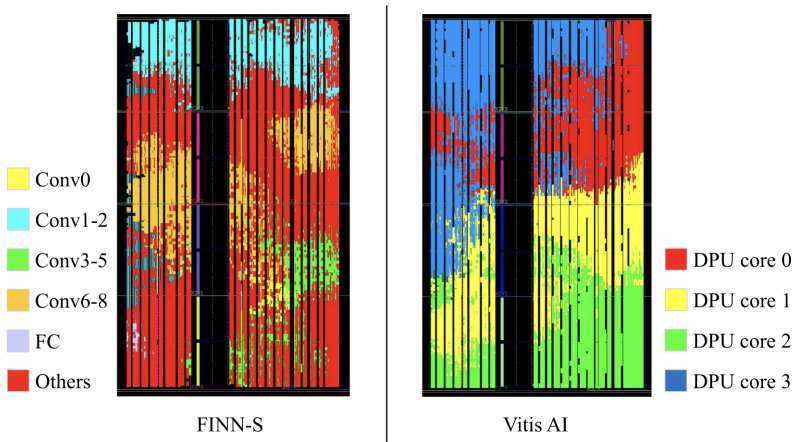
- ▶ DL tienen requerimientos computacionales.
- ▶ **Limitaciones de CPU y GPU en consumo energético y latencia.**
- ▶ FPGA y SoC como plataformas reconfigurables y eficientes.
- ▶ AMD Vitis AI es un ecosistema integral para desplegar modelos de IA.



Speed up the deployment path with Vitis AI 2.5
Adaptive SoC & FPGA Support

- ▶ Componentes principales:
 - Vitis AI Compiler
 - Vitis AI Optimizer
 - Vitis AI Runtime
 - DPU (Deep Learning Processing Unit)
- ▶ Integración con frameworks de IA: TensorFlow, PyTorch, ONNX.
- ▶ Soporte para FPGAs y SoCs AMD.



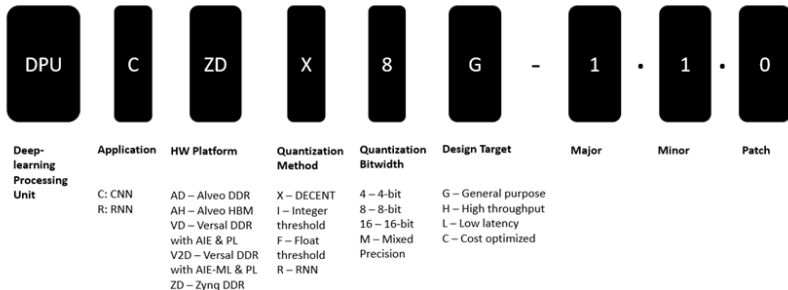


The place and route results of our FINN-based and Vitis-AI-based accelerators³

³F.Hamanaka, T.Odan, K.Kise y T.V. Chu, "An Exploration of State-of-the-Art Automation Frameworks for FPGA-Based DNN Acceleration", IEEE Access, vol.11, pp.5701-5713, Ene.2023

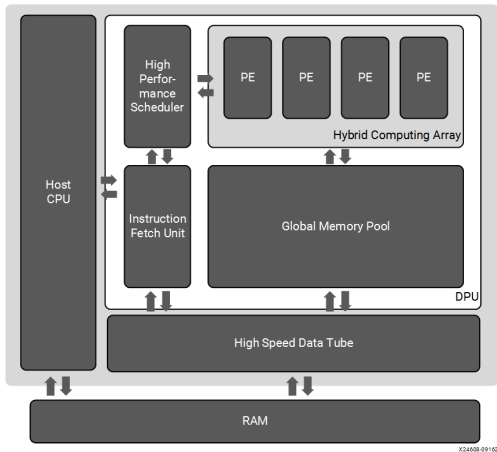
Deep-Learning Processing Unit (DPU)

- **Motor programable** optimizado para inferencia de modelos DL.
- **IP cores parametrizables**, preimplementados en hardware (no requiere **place & route**).
- Ejecuta instrucciones especializadas de Vitis AI para CNNs modernas: VGG, ResNet, GoogLeNet, YOLO, SSD, MobileNet, entre otras.
 - **Edge**: Zynq-7000, Zynq UltraScale+ MPSoC, Kria KV260.
 - **Cloud**: Versal v tarjetas Alveo (U50. U200. U250. U280).



DPU[Plataforma] [Cuantización] [Bits] [Objetivo] - [Mayor] . [Menor] . [Parche]

Zynq UltraScale+ MPSoC: Optimized for DPUCZDX8G

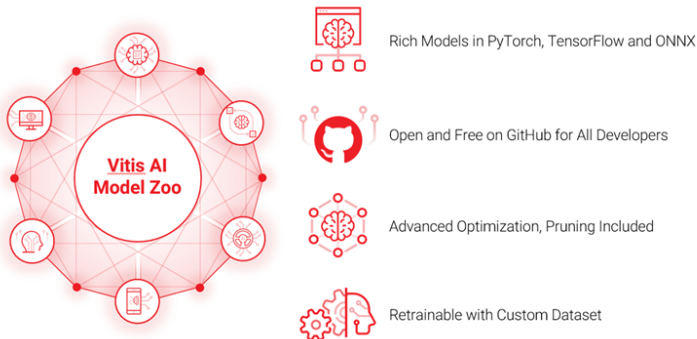


- ▶ Se integra como **IP core** en la **lógica programable (PL)** con conexión directa al **sistema de procesamiento (PS)**.
- ▶ Configurable por el usuario:
 - Ajuste de uso de recursos PL (DSPs, BRAM, LUTs).

- ▶ **Inferencia (CNN)** ejecutando operaciones **MAC** masivamente en paralelo (*array* de Processing Elements (PEs) tipo systolic).
- ▶ **Instruction Fetch Unit** lee el microcódigo generado por Vitis AI (conv, pool, activaciones, entre otros).
- ▶ **High-Performance Scheduler** Dirige cómputo y transferencias para mantener el hardware ocupado.
- ▶ **Global Memory Pool** almacena y **reusa** pesos/activaciones con doble *buffer* (ping-pong), reduciendo accesos a DDR.
- ▶ **High-Speed Data Tube** mueve *tiles* entre **RAM** y la DPU con alto ancho de banda.
- ▶ **Host CPU** envía trabajos, configura tensores y recoge resultados.
- ▶ **El Hybrid Computing Array...** organización de la DPU, que combina distintas funciones optimizadas para DL.
- ▶ *Mayor throughput y menor latencia al minimizar accesos a memoria externa y maximizar paralelismo interno.*

Flujo: Host CPU → RAM ↔ Data Tube → DPU (IFU + Scheduler + PEs + Mem Pool) → RAM → Host CPU

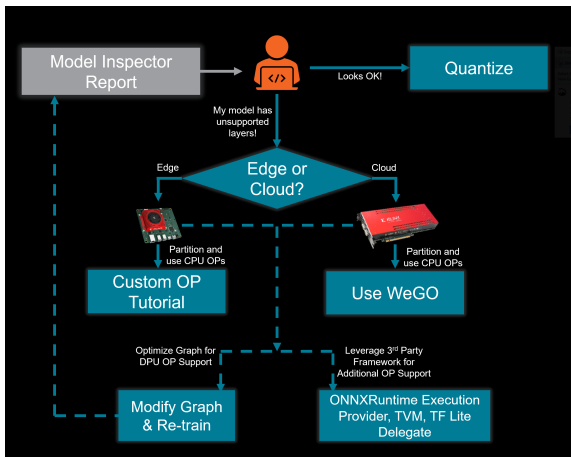
Model Development



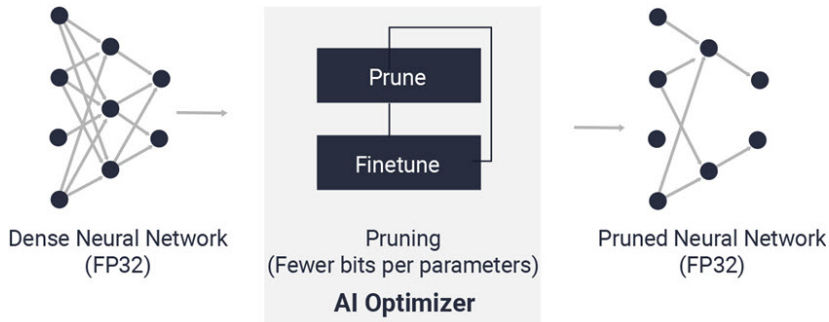
- ▶ Colección de **modelos de DL optimizados** para plataformas Xilinx.
- ▶ Permite **acelerar el despliegue de inferencia** sin entrenar desde cero.
- ▶ Modelos preentrenados para diversas aplicaciones:
 - ADAS y conducción autónoma, video vigilancia, y robótica⁴.

⁴<https://github.com/Xilinx/Vitis-AI>

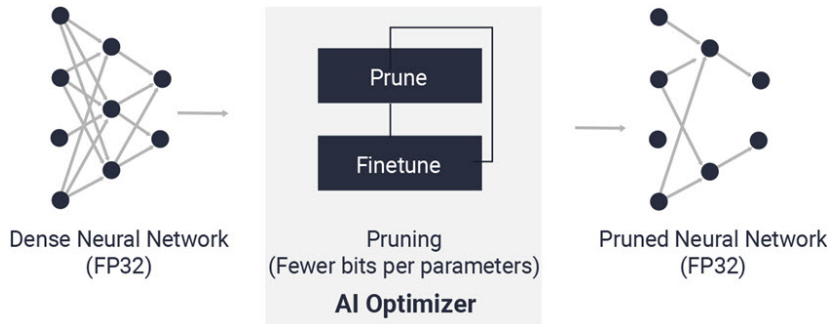
Vitis AI Model Inspector



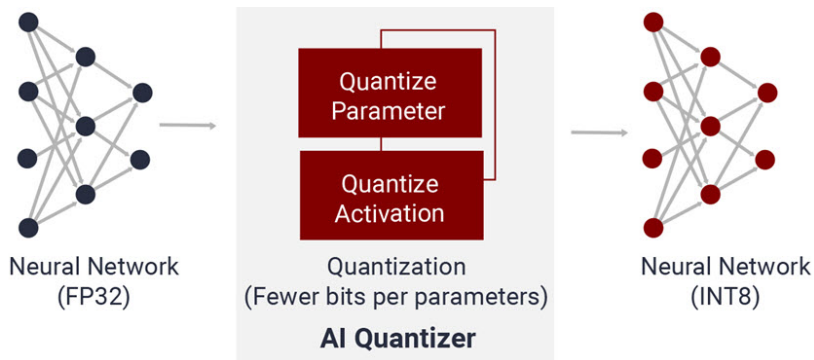
- ▶ AMD busca dar soporte a una amplia variedad de arquitecturas y entregar ANNs de referencia, pero sólo algunos operadores están soportados para la aceleración en la DPU.
- ▶ Existen requisitos específicos en el orden de las capas que permiten el despliegue de modelos en Vitis AI.
- ▶ En fases tempranas de desarrollo, se recomienda usar el **Vitis AI Model Inspector** como **sanity check** para confirmar la compatibilidad de los operadores.



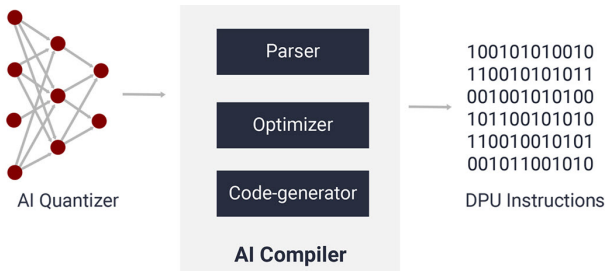
- ▶ **Herramienta de compresión de redes** (principalmente **pruning estructurado** por canales/filtros... no pesos sueltos sino toda la estructura).
- ▶ Mantiene la **precisión** mediante *fine-tuning* tras el pruning.
- ▶ Produce modelos más **eficientes** para desplegar en la **DPU (INT8)**, priorizando **pruning estructurado** que el hardware aprovecha mejor.



- ▶ (1) **Base FP32**: entrena el modelo original.
- ▶ (2) **Análisis de sensibilidad**: decide *ratios* de pruning (global/por capa) y capas a excluir
- ▶ (3) **Pruning estructurado + fine-tuning** iterativo para recuperar accuracy.
- ▶ (4) **Quantización** (PTQ/QAT) con Vitis AI Quantizer a **INT8**.
- ▶ (5) **Compilación** para **DPU** (Vitis AI Compiler)

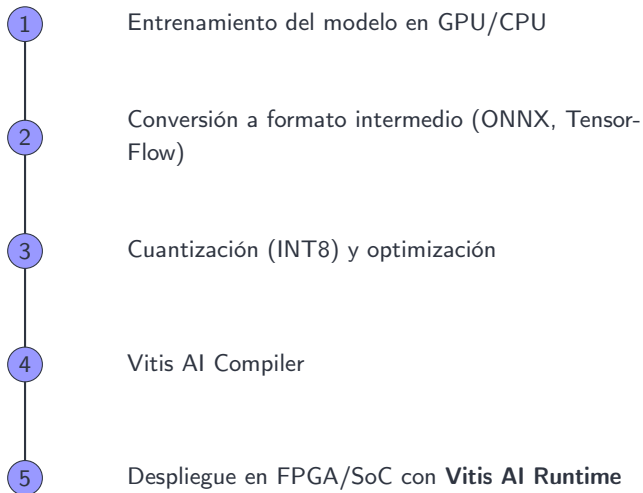


- ▶ Usa calibración con un conjunto representativo (PTQ) y puede emplear **QAT** para recuperar precisión.
- ▶ Modelo DL **INT8** listo para la **DPU**.
- ▶ Menos uso de **memoria** y **ancho de banda** reducido, **latencia** menor y mayor **throughput**/eficiencia energética.
- ▶ Ligera caída de precisión con PTQ... **QAT** ayuda a mantener la métrica objetivo.



- Recibe el modelo **cuantizado** (INT8) del *Vitis AI Quantizer*.
- **Parser**: lee el grafo, verifica ops soportadas por la **DPU**, y delega ops no soportadas al **CPU**.
- **Optimizer**: Fusiona capas (ej., *Conv+BN+ReLU*), elimina nops/constantes, hace **tiling** y **planifica** para encajar en la memoria on-chip y maximizar el throughput.
- **Code-generator**: Mapea el grafo a la **ISA**⁵ de la **DPU**, asigna buffers (on-chip/DDR), reordena pesos/activaciones y emite el **flujo de instrucciones** + metadatos para **runtime** (VART).
- **Instrucciones DPU + pesos/escala, listos para ejecutar en la tarjeta.**

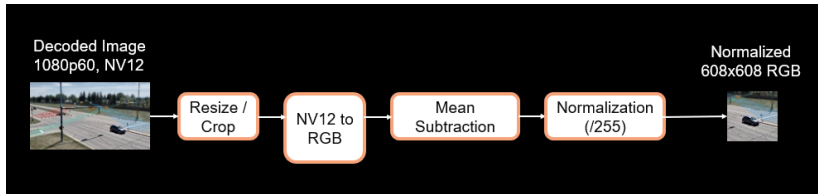
⁵Instruction Set Architecture... Set de instrucciones nativas que entiende el hardware; en la DPU define cómo ejecutar operaciones de DL



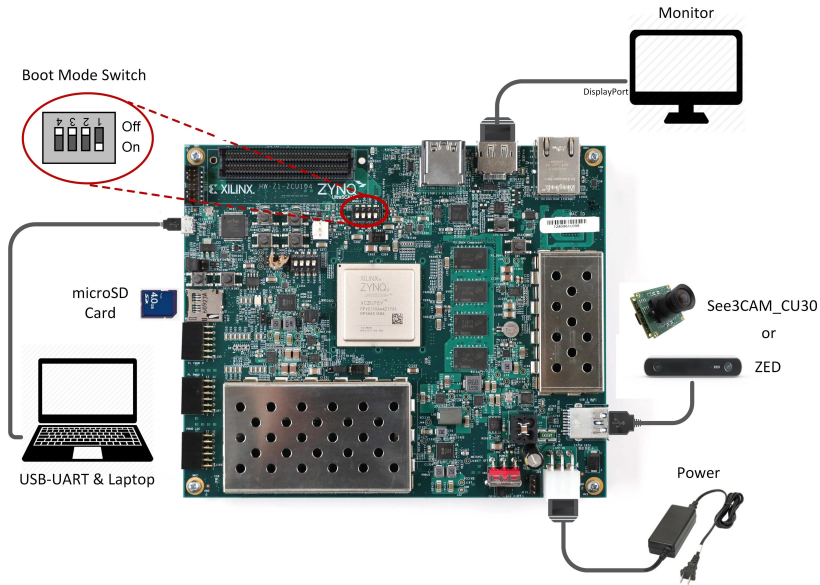
Model Deployment

Vitis AI Runtime (VART)

- ▶ Conjunto de **API de bajo nivel** para integrar el DPU en aplicaciones.
- ▶ Construido sobre **Xilinx Runtime (XRT)**⁶.
- ▶ Proporciona un runtime unificado para **Data Center** y **Embedded**.
- ▶ **Características principales:**
 - Envío asíncrono de trabajos al DPU.
 - Recolección asíncrona de trabajos desde el DPU.
 - APIs disponibles en **C++** y **Python**.
 - Soporte para **multi-threading** y **multi-process**.



⁶ Capa de software que gestiona el hardware de la FPGA y permite ejecutar kernels y mover datos entre la CPU y el acelerador.



► Guide KV260/ZCU102/ZCU104 boards

- Compilación de ResNet50 para DPU.
- Comando típico:

Vitis AI Compiler

```
vai_c_tensorflow -frozen_pb model.pb  
-arch ./arch.json -output_dir ./compiled_resnet
```

- Ejecución con VART (Vitis AI Runtime).